

Sprint Planning Document

AI Knowledge Assistant (RAG) | 4 Sprints | 8 Weeks

1. Project Overview

Product: A chat assistant that answers questions from a company's internal documents (policies, FAQs, manuals) and cites its sources.

Business goal: Cut the time staff spend searching for information, and avoid wrong or made-up answers.

Success criteria:

- At least 85% of answers are grounded in the retrieved documents
- Every answer includes a source citation
- Responses take under 5 seconds
- The assistant says "I don't know" when the documents don't cover the question

2. Team Roles

Role	Responsibility
Product Owner	Defines the backlog, priorities and acceptance criteria
Scrum Master	Runs the ceremonies, removes blockers, tracks velocity
ML/AI Engineer	Embeddings, retrieval, prompts, LLM integration
Backend Engineer	API, vector database, authentication
Frontend Engineer	Chat interface
Data Engineer	Document ingestion and cleaning
QA / Responsible AI Lead	Evaluation, hallucination and bias tests, privacy review
DevOps	CI/CD, deployment, monitoring

On a small team, one person can hold several roles.

3. Architecture Design



Orchestrator step	What it does
Query rewriting	Cleans up and expands the user's question
Retriever + Vector DB	Hybrid search over documents embedded by the ingestion pipeline
Reranker	Orders retrieved chunks by relevance
LLM	Answers using the prompt plus retrieved context only
Guardrails	PII filter and grounding check before the answer is returned
Answer + citations	Returned to the user; logged for monitoring

Main components:

- **Ingestion pipeline:** parse, clean, chunk and embed documents
- **Retrieval layer:** hybrid search (keyword plus vector) with a reranker
- **Generation layer:** an LLM prompted to answer only from the retrieved context
- **Guardrails:** PII redaction and a check that the answer is grounded
- **Observability:** logs, latency, user feedback and evaluation dashboards

4. Agile Setup

- **Sprint length:** 2 weeks, 4 sprints in total
- **Ceremonies:** sprint planning, daily 15-minute standup, sprint review, retrospective
- **Definition of Done:** code reviewed and merged, tests passing, documented, and demoed

5. Git Collaboration Workflow

- **Branches:** main (protected, always deployable), develop (integration), and feature/<ticket-id>-short-name for each task
- **Commits:** small and descriptive, for example feat: add document chunker
- **Pull requests:** at least one reviewer, CI must pass, and each PR links to its ticket
- **Merges:** squash-merge into develop, and release by merging develop into main at the end of each sprint
- **Conflicts:** rebase on develop daily and keep features small

6. Task Priority Table

Complexity: L = Low (about 1 day or less), M = Medium (2 to 3 days), H = High (4 days or more). **Business value:** how much the task contributes to the working product.

ID	Task	Complexity	Business Value	Depends On	Priority	Sprint
T1	Define requirements, user stories, success metrics	L	High	None	1	1
T2	Set up Git repo, branching rules, CI pipeline	L	High	None	1	1
T3	Finalize architecture and choose tech stack	M	High	T1	2	1
T4	Collect and clean sample document set	M	High	T1	2	1
T5	Build ingestion pipeline (parse, chunk, embed)	H	High	T3, T4	3	2
T6	Set up vector database	M	High	T3	3	2
T7	Build retrieval (hybrid search)	H	High	T5, T6	4	2
T8	Design prompts and integrate LLM	M	High	T7	5	2
T9	Build backend API and authentication	M	High	T3	4	2
T10	Build chat UI with citations	M	Medium	T9	6	3
T11	Create evaluation set and metrics (grounding, accuracy)	M	High	T8	6	3
T12	Add reranker and query rewriting	M	Medium	T7	7	3
T13	Add guardrails (PII filter, "I don't know" behavior)	M	High	T8	6	3
T14	Bias, privacy and hallucination testing	M	High	T11, T13	7	3
T15	Integration and end-to-end testing	M	High	T10, T13	8	4
T16	Deploy to staging, then production	M	High	T15	9	4
T17	Monitoring, logging and feedback button	L	Medium	T16	10	4
T18	Documentation and user guide	L	Medium	T15	10	4
T19	Final demo and retrospective	L	Medium	T16	11	4

Why this order: tasks with high business value and no blockers (T1, T2) go first. Tasks that unblock many others (T3, T5, T7) come next. Nice-to-haves such as the reranker (T12) and monitoring (T17) come later, so the project is still usable if time runs short.

7. Sprint Plan

Sprint	Goal	Tasks
Sprint 1 Weeks 1-2	Foundation: plan, repo, architecture, data	T1, T2, T3, T4
Sprint 2 Weeks 3-4	Core engine: a working question-to-answer flow	T5, T6, T7, T8, T9
Sprint 3 Weeks 5-6	Quality and safety: UI, evaluation, guardrails, responsible AI tests	T10, T11, T12, T13, T14
Sprint 4 Weeks 7-8	Release: testing, deployment, docs, demo	T15, T16, T17, T18, T19

Risks and mitigations:

- Poor document quality: start cleaning in Sprint 1
- Hallucinations: build evaluation and guardrails early, in Sprint 3
- Sprint 2 is heavy (two High tasks): move T9 earlier if needed, and keep T12 as the spare to drop
- LLM cost or latency: cache results and test with smaller models first

8. Project Timeline

Sprint / Tasks	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5	Wk 6	Wk 7	Wk 8
Sprint 1: T1-T4 Plan, repo, architecture, data								
Sprint 2: T5-T9 Ingestion, retrieval, LLM, API								
Sprint 3: T10-T14 UI, eval, guardrails, RAI tests								
Sprint 4: T15-T19 Test, deploy, docs, demo								

Milestone	Week	What is delivered
M1	2	Approved architecture, repo and CI, clean data
M2	4	MVP: ask a question and get a cited answer
M3	6	Beta with UI, guardrails and evaluation results
M4	8	Production release, documentation and demo